arXiv:2409.20326v3 [cs.MA] 18 Mar 2025

MARLadona - Towards Cooperative Team Play Using Multi-Agent Reinforcement Learning

Zichong Li, Filip Bjelonic, Victor Klemm, and Marco Hutter

***Abstract*— Robot soccer, in its full complexity, poses an unsolved research challenge. Current solutions heavily rely on engineered heuristic strategies, which lack robustness and adaptability. Deep reinforcement learning has gained significant traction in various complex robotics tasks such as locomotion, manipulation, and competitive games (e.g., AlphaZero, OpenAI Five), making it a promising solution to the robot soccer problem. This paper introduces MARLadona. A decentralized multi-agent reinforcement learning (MARL) training pipeline capable of producing agents with sophisticated team play behavior, bridging the shortcomings of heuristic methods. Furthermore, we created an open-source multi-agent soccer environment. Utilizing our MARL framework and a modified global entity encoder (GEE) as our core architecture, our approach achieves a 66.8 % win rate against HELIOS agent, which employs a state-of-the-art heuristic strategy. In addition, we provided an in-depth analysis of the policy behavior and interpreted the agent's intention using the critic network.**

I. INTRODUCTION

Recent advancements in deep reinforcement learning (RL) have made it possible to solve some of the most challenging robotic tasks such as object manipulation [1], locomotion [2], and high-level navigation tasks [3]. However, many real-world problems require cooperation among multiple robots, requiring RL agents to learn optimal behavior while coexisting with other agents, with either conflicting or common objectives requiring negotiation and communication. As a result, this intricate interplay of strategies introduces additional complexity, making multi-agent system (MAS) more challenging than single-agent systems.

As one of the most popular MAS, robot soccer has been the grand goal of robotic and AI research. Competitions such as RoboCup (RC) [4] and IEEE Very Small Size Soccer [5] have inspired generations of researchers to push the state-of-the-art (SOTA) in robotic hardware and algorithms. Traditionally, the high-level strategies of soccer agents are often hierarchically separated from the rest and tackled with sophisticated decision trees crafted by human experts. But robot soccer as a pure learning problem has gained significant attention in recent years [6], [7]. Existing literature in soccer MARL can be roughly categorized into creating environments and proposing training approaches.

A. Related Work

Several soccer environments have already been proposed by literature such as Google research football [8], MARL2DSoccer [9], rSoccer [5], Humanoid football in MoJoCo [10] and the RC Simulation2D server [4]. However, besides lacking GPU support, many existing MARL soccer environments are kinematic simulators [9], [5], [4]. The lack of physics creates an additional sim-to-real gap, making transferring the learned strategies to real robotic platforms harder. On the other hand, MARL environments like [8] are great for comparing novel algorithms by providing a standardized framework. Still, its game-inspired actions, i.e., abstracted action like "long pass", make it even less applicable. Most RC teams also have their own customized multi-agent soccer simulators [11], [12], [13] tailored and fine-tuned for their specific robotic platforms.

While existing approaches for MARL soccer often tackle the multi-agent strategy as a stand-alone MARL problem [9], [14], [15]. Recent works from DeepMind [6], [7] have demonstrated an approach that solves the full soccer problem end-to-end in a 3D environment. Both of these works utilized a combination of imitation learning and individual skill training to obtain low to mid-level motor skills. Then, they trained with MARL in a self-play setting to get the final policy. While emerging team play was observed in [6], they only demonstrated within a 3v3 game setting. For games with more agents, it is more common to simplify the soccer problem into 2D and train high-level policies [14], [15], [9], [5], [16] which assume the existence of motion interfaces such as walking velocity and kicking commands. Existing work commonly trains decentralized policies [9], [17] to avoid the exploding state space with growing agent

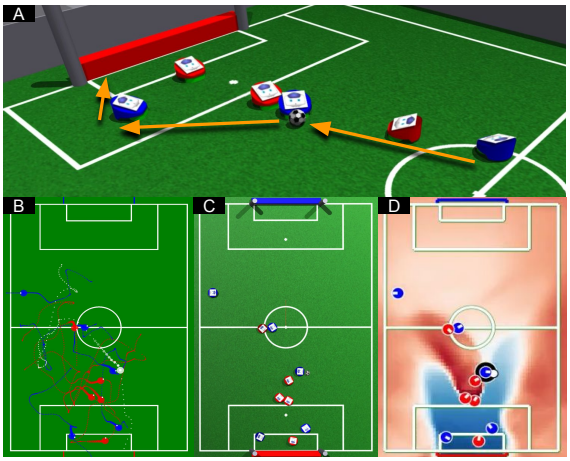


Fig. 1. An illustration of our MARL environment (A) and various top-down views of a 5v5 game (B-D). (B) The trajectory visualizer depicts the general game dynamic. (C) The corresponding default top-down view. (D) The corresponding ball position critic value heat map.

This paper has been accepted for publication at the IEEE International Conference on Robotics and Automation (ICRA) 2025.
All authors are with the Robotic Systems Lab, ETH Zürich, 8092 Zürich, Switzerland.

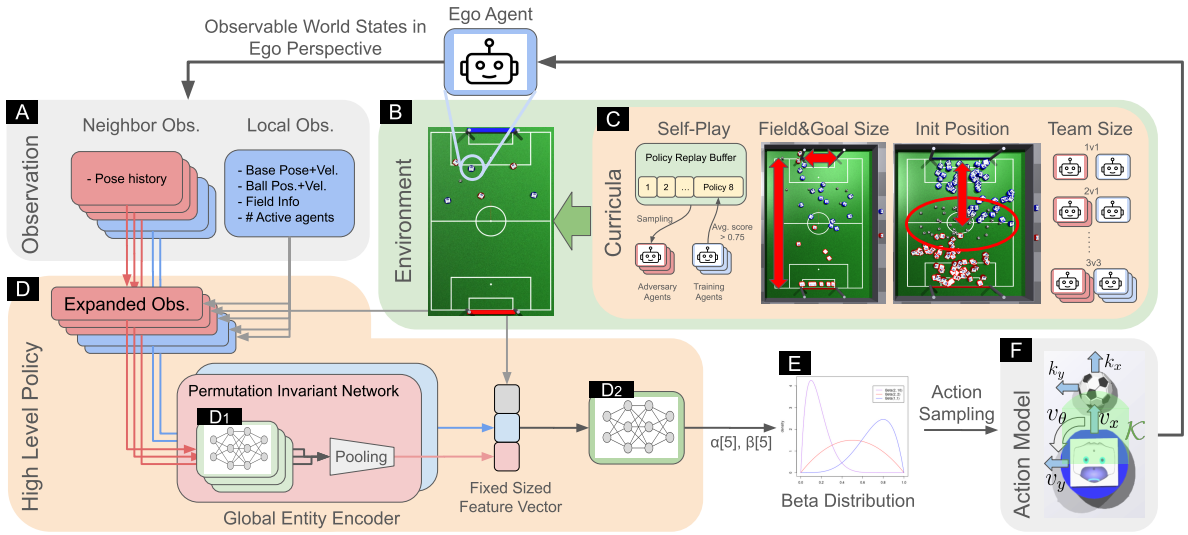


Fig. 2. An overview of our system. (A) The ego perspective observation from the opponents (red), teammates (blue), and local observation. (B) The soccer environment. (C) Various curricula we adopted during training. (D) The architecture of our policy network. (D1) Encoders with shared weights. (D2) Policy network. (E) The distribution we used for action sampling. (F) The action model of our soccer agent.

number. For instance, Smit et al. [9] used an attention mechanism and utilized their customized MARL2DSoccer environment for simulation. Lin et al. [17] used Google research football directly and presented the TiZero, which utilized a combination of observation encoder and LSTM mechanism with a multi-stage self-play curriculum. They achieved SOTA performance, but their game-inspired actions make their policy incompatible with actual robotic systems.

For many MAS, permutation invariant with respect to neighboring agents is a desirable property. An indirect way to achieve this is to augment the data with the permutated state, similar to learning symmetry [18]. However, it is more desirable to have it directly as a network property. Various forms of permutation invariant networks have been proposed in the literature, e.g., Attention mechanism [19], PointNet [20], deep sets [21] etc. Based on the PointNet, An et al. [14] proposed the GEE in which set inputs of the same type, called entities, are passed through a shared multilayer perceptron (MLP) allowing a list of local feature encodings to be obtained. The max pooling is then conducted over the set of local feature encodings and the resulting global entity feature, which is permutation invariant for set inputs.

B. Contributions

While existing training approaches often demonstrate improved quantitative results against some heuristics [9], [5], [16], none of them, to our best knowledge, demonstrated quality team play in MARL soccer for more than 3v3 games. To address this open research problem, we introduce MARLadona, a novel framework for training high-level multi-agent policies using end-to-end RL.

Our key contributions include:

- A new customizable open-source MARL soccer environment based on Isaac Lab[1].
- Introduced an improved GEE for training policies that achieve effective team play for games up to 11v11.
- Provided a comprehensive analysis of the policy behavior and benchmarked its performance against HELIOS[2], a SOTA scripted soccer agent.

II. METHOD

A. Overview

Our work aims to train a generic decentralized soccer policy that demonstrates effective and dynamic team play. To achieve this, we designed a multi-agent soccer environment on top of Isaac Gym to leverage its realistic physics and GPU acceleration and utilized this framework in combination with a novel approach (Fig. 2) to train the soccer policy. We address the sparse reward problem of soccer MARL with different curricula (C) and adopted a modified version of the GEE architecture [14] as our actor (D) to effectively handle permutation and number changes among both teammates and opponents. Moreover, our approach aims to include only the minimum for observation states and rewards. By using the PPO algorithm [23], we trained our lightweight network on a consumer desktop (single RTX 2060 GPU) and achieved quality team play within just a few hours of training.

B. Markov Game

Different from traditional single-agent RL problems, MARL the problem is often modeled as a Markov game (MG) [24] which is a generalization of the markov decision process (MDP) to multiple agents. A MG can be formally defined as a tuple $\langle \mathcal{I}, \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where $\mathcal{I}$ denotes the set of agents, $\mathcal{S}$ is the environment state, $\mathcal{A} = \{a_i\}_{i=1}^{N}$ is the joint action of all agents, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ the transition probability, $\mathcal{R} = \{r_i\}_{i=1}^{N}$ with $r_i : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the reward function of agent i and γ as the discount factor. At each time step t, the state $\mathbf{s} \in \mathcal{S}$ evolves according to the

[1]https://github.com/leggedrobotics/marladona-isaac-lab
Note that the original work in this paper was based on Isaac Gym.

[2]For clarity, we use HELIOS to refer to [22]

transition probability $\mathcal{P}$ and the joint action $\mathbf{a} \in \mathcal{A}$. Each agent i then receives their respective reward r_i from the environment as feedback.

Often in MARL, the global environment states are not fully observable, leading to the partial observable Markov game (POMG) [25]. The additional $\mathcal{O}$ is used to denote the observable states of the environment. In case all agents share the same reward function, then a cooperative POMG can be considered as a decentralized partially observable Markov decision process (Dec-POMDP) [26].

For this paper, we adopted the centralized training and decentralized execution (CTDE) paradigm due to its scalability and robustness to varying agent numbers and access to privileged information. In CTDE, each agent has its policy $\pi_i : \omega_i(\mathcal{O}) \rightarrow a_i$, which maps its local observation to its action a_i. ω_i is the observation function of agent i. Denoting the additional set of observation function as $\Omega = \{\omega_i\}_{i=1}^{N}$, our final problem can be modeled as a decentralized partially observable Markov game (Dec-POMG) defined by the tuple:

$$\langle \mathcal{I}, \mathcal{S}, \mathcal{A}, \mathcal{O}, \Omega, \mathcal{P}, \mathcal{R}, \gamma \rangle \tag{1}$$

C. Environments and Agent Modeling

Our MARL soccer environment (Fig. 2 B) is built on top of Isaac Gym [27], allowing us to simulate thousands of games in parallel. Each environment contains a ball and two teams with a random number of agents, and all actors are uniformly spawned within their designated area. A game terminates when a goal is scored or after 30 seconds, which is empirically chosen to give the agents enough time to score. The ball respawns above the borderline upon leaving the playing field, and a physical wall is placed at a short distance around the playing field to prevent escaping agents. Furthermore, the environment registers events such as passes, ownership[3] losses, etc. Only the blue agents are used for training, while the red agents are controlled by bots or old policies, depending on the scenario. For clarity, we will refer to the blue agent as **trainee** and the red agent as **adversary**.

A floating base with a cylinder as its collision body is used for the agent modeling. At each simulation step, each agent receives an action command of the form $(v_x, v_y, v_\theta, k_x, k_y)$ (Fig. 2 F). All actions represent velocity commands[4] which is then tracked by a PD controller outputting a corresponding force and torque for the simulator. The x and y component of the translation commands (base: v_x, v_y) and (kick: k_x, k_y) are remapped using Eq. 2 to map the rectangular control space into the unit disk. Note, the agent's kick commands are only in effect when the ball is within its kickable area $\mathcal{K}$.

$$x_{new} = x \cdot \sqrt{1 - \frac{y^2}{2}} \quad , \quad y_{new} = y \cdot \sqrt{1 - \frac{x^2}{2}} \tag{2}$$

D. Curricula

To reduce the initial exploration complexity and accelerate the learning process, we included various types of curriculum during training (Fig. 2 C). Inspired by terrain curriculum [28], we added different field and goal size levels to increase the initial scoring chance and adjusted the environment level depending on the outcome of the previous game.

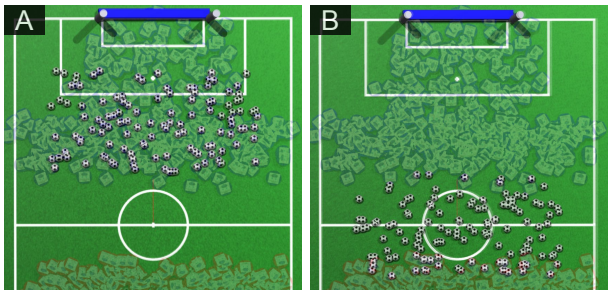


Fig. 3. Initial position curriculum depended on the current policy performance. The ball's initial distribution is adjusted toward the blue side for lower levels to enhance trainees' chances of gaining ball procession. The agent's initial distribution, on the other hand, is kept constant.

Self-play is another approach commonly applied in competitive RL problems [29], [7], [8], [9], [30]. It allows our trainees to adapt to different adversaries and incentivizes them to learn more generalized strategies. Various forms of implementation can be found in the literature, such as League training, Polyak averaging [9], or sampling from past policies [7]. We adopted a replay mechanism that buffers up to 8 policies. During training, the current trainee policy is added to the adversary buffer when it achieves an $75\,\%$ average win rate against all of its adversaries. The adversaries then sample from this buffer and run the policy in inference mode. During sampling, we apply the same policy per team, as cooperation quality might deteriorate due to strategy mix-ups.

Unlike many other MARL problems, the initial position of the agents and ball is critical as being closer to the ball directly implies ownership, which is a significant advantage in soccer. Therefore, we adopted different curriculum levels to the initial position as well. In our approach, agents are unaffected by the curriculum level and just spawned uniformly within their designated area with direction sampled from $(\pi, -\pi)$. The ball is biased deeper into the blue side for lower curriculum levels to increase ball ownership probability for the trainee team (Fig. 3).

Finally, we also want the trainees to learn and adapt their strategies accordingly, depending on the team sizes. Since training directly with the maximum agent number can be counterproductive, we implemented a resampling mechanism that allows the environment to dynamically re-configuration its team composition before an episode begins. This enables us to adapt the maximum number of players according to the policy performance. Due to computational constraints, we limited our training to a maximum of 3v3 games.

E. Observations and Rewards

An overview of the observation is summarized in Tab. I. Rows 1-6 are local observations, and rows 7 and 8 show the neighbor observations. Both actor and critic use the same observation states but slightly different noise configurations. Note that all observation types are transformed into the perspective of the ego agent. The option column provides ad-

[3] An agent is assigned with ball ownership if it is within $0.25\,\mathrm{m}$, the closest from his team, and no opponent fulfills the first condition.

[4] Velocity commands as it is widely used to interface with robotic platforms.

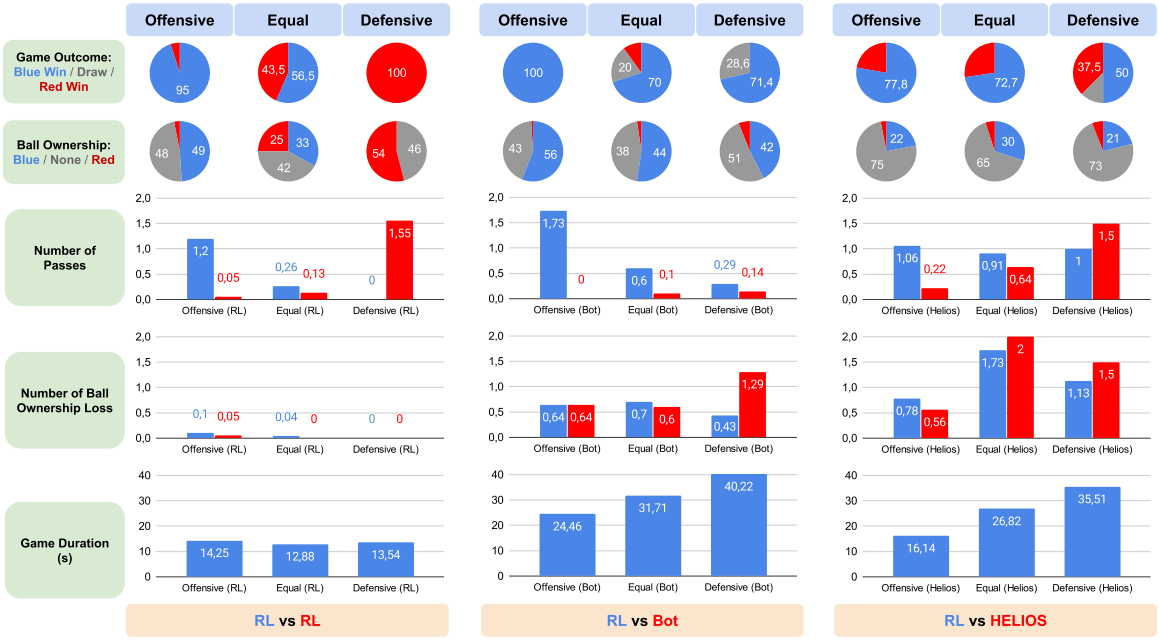


Fig. 4. An overview of our evaluation results conducted for a 3v3 game for three different scenarios (Offensive, Equal, Defensive) against three different adversaries (RL, Bot, HELIOS). The collected average statistics (game outcome (%), team ball ownership (%), the number of successful passes and ball ownership losses, and game duration) are depicted in different rows. Our trainee policy (Blue) achieved clear dominance against all adversaries (besides itself) in all scenarios. The trainees won 66.8 % (averaged over all three scenarios) of all games against HELIOS.

TABLE I

OBSERVATION OVERVIEW.

Observation name	Noise actor/critic	Dimensions	Options
Local Observations			
1) Base pose	0.002 / 0	4	W, E, N
2) Base velocity	0.005 / 0	3	W
3) Ball position	0.002 / 0	2	W, N
4) Ball velocity	0.005 / 0	2	W
5) Field info	0 / 0	5	N
6) # active agents	0 / 0	2	
Neighbor Observations			
7) Teammate poses	0.002 / 0	$4 \cdot H \cdot N_T$	W, E, N
8) Opponent poses	0.002 / 0	$4 \cdot H \cdot N_O$	W, E, N

TABLE II

REWARD OVERVIEW.

Reward name	Description	Scale	Shared
Sparse Rewards			
1) Score	+1 if wins, -1 if loses	100	True
2) Ball outside field	-1 if ball leaves the field.	1	True
3) Collision	-1 if collision (except ball)	1	
Dense Rewards	(Removed later)		
4) Ball2goal velocity	$\|\|v_{ball2goal}\|\|$	2	True
5) Base2ball velocity	$\|\|v_{base2goal}\|\|$ if ball is far	0.5	
6) Ball direction	$\exp(-(\theta_{base2ball}/0.4)^2)$	0.025	

TABLE III

NETWORK CONFIGURATION.

	Input	Hidden	Output	Activation
Encoder network	27	64, 32	16	ELU
Policy network	35	128, 128, 128	10	ELU

ditional configuration details. (W) indicates the observation is in the world frame, (E) denotes the rotation component θ is expanded into $\sin(\theta)$ and $\cos(\theta)$. (N) means we normalize the observation with the field length and width. And H shows the number of included past time steps. The field info contains auxiliary information about the x and y position of the field line border and goal width. For the neighbor observations, the observation function Ω is implemented as a filter mask that passes the N_{max} closest agents' state, which is kept at three during training. For testing with higher agent numbers, e.g., 5v5 and above, we use $N_{max} = 5$ and also limit the value of observation 6 in Tab. I to three active agents, avoiding scalability issues.

Tab. II provides an overview of our rewards. Rows 1-3 are our main sparse rewards, and rows 4-6 show the dense rewards that get permanently removed the first time the trainee policy reaches 75 % win rate against its adversaries. The dense rewards are only added to provide initial guidance by incentivizing the trainees to approach the ball if neither the ego agent nor his teammates have ball ownership (reward 4), bring the ball closer to the goal (reward 5), and keep the ball within its kickable area $\mathcal{K}$ (reward 6). The shared column indicates which of them is shared with the whole team.

F. Network Architecture

MAS often requires agents to have invariant strategies with respect to neighboring agents while dynamically adapting to changes in agent number. We achieved this using a similar architecture as the GEE [14] (Fig. 2 D). One key difference to the original GEE architecture is the expanded observation formed by concatenating each neighboring agent's observation with the local observation. This expanded observation is then forward fed into the encoder network (D1), allowing the shared encoder to extract information with additional context of the ego agent. The most prominent feature of the resulting local entity feature is max pooled across each agent type. The final input for the policy network (D2) can be obtained by merging the fixed-size global entity feature

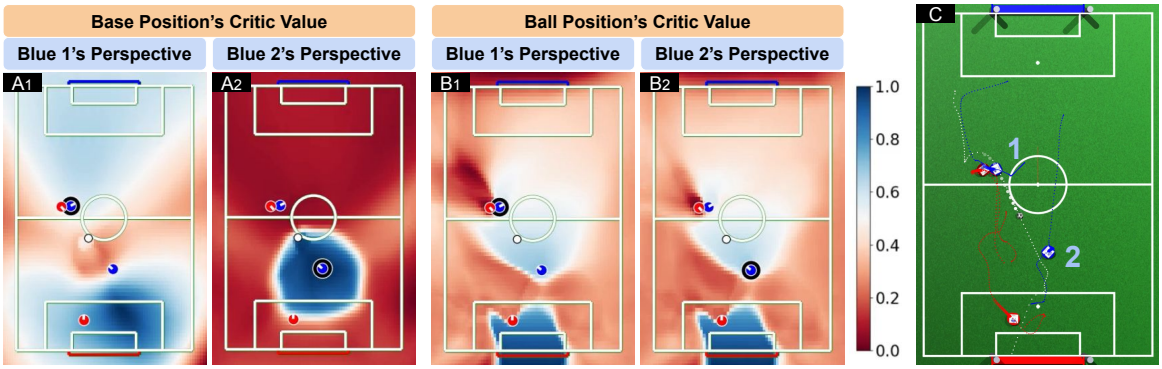


Fig. 5. An illustration of the critic values from a 2v2 (RL vs RL) game as a heat map (Res. 80 × 80). The plots are acquired by varying the base positions of the trainees (A1, A2) and the ball position (B1, B2) over the whole field while keeping the other observations fixed. The position of the trainees (blue), adversaries (red), and ball (white) are represented by their respective colored dots, and the large black circle indicates which of the trainees the heat map belongs to. Furthermore, C provides the corresponding default top-down view overlaid with motion trajectories to provide additional information about the current game dynamic. The blue areas on these heat maps indicate where the agents want themselves and the ball to be.

of the teammates and opponents, which can be concatenated with the local observation. The layer size configuration of both the encoder and policy network are summarized in Tab. III. Similar to [14], we use beta distribution to adhere to the constraint of the velocity command, which requires the policy net to predict a (α, β) value pair per action.

III. EXPERIMENTAL SETUP

A. Evaluation Methodology

We validated our trainee policy against three adversaries using three scenarios in a 3v3 setting. Over 600 seconds of simulated gameplay (6~20 game sets), we tracked various performance metrics, including game outcome, ball procession, the average number of passes, ball losses, and game duration. Sec. III-B will provide additional details about the adversaries. Identical to the training setup, we spawn agents uniformly in their designated areas. While the spawning distribution is identical between the two teams in all scenarios, we biased the ball spawn location deep into the blue side for **Offensive**, into the red side for **Defensive** and kept them in the field center for **Equal** scenario.

Additionally, we provide insight into the trainee's behavior directly with trajectory rollouts (Fig. 6 and 7) and value function of the base and ball position (Fig. 5).

B. Adversaries

For the adversaries, we have the trainee itself (**RL**), a simple heuristic bot (**Bot**), and lastly the HELIOS agent as a SOTA benchmark (**HELIOS**). The heuristic bot is a role-based scripted bot that assigns the closest agent with greedy soccer logic, i.e., approaching the ball and immediately kicking toward the goal once the ball becomes kickable. It also has a goalkeeper and defender logic. It is a simple yet effective adversary but far from sufficient as a benchmark. Proper benchmarking, however, is currently very difficult due to the lack of unified MARL soccer frameworks. Most existing MARL policies are embedded in other custom simulators, which lack interoperability. For our benchmark, we created an interface server using the RC Simulation2D protocol [31], allowing us to compete against algorithms from other Simulation2D teams within our Isaac Gym environment. We chose HELIOS since it is currently the best-performing framework in the RC Simulation2D, winning a total of 6 world championships in the past. To reduce the handicap introduced by the sim-to-sim gap, we replicated the movement and kick model inside Isaac Gym to the best of our ability.

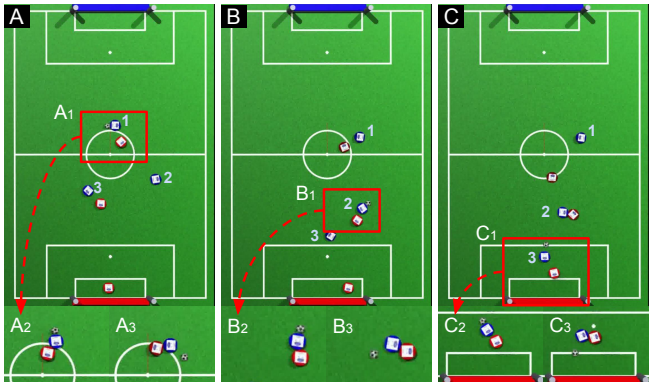


Fig. 6. A top-down view sequence depicting typical trainee (blue) behaviors against bots in a 3v3 game setting. A, B, and C provide an overview of the global behavior, such as positioning, while A1-3, B1-3, and C1-3 provide a zoom-in view showcasing numerous local ball-handling skills.

IV. RESULTS AND ANALYSES

A. Quantitative Results

Fig. 4 provides an overview of the evaluation result where the trainee policy performs against different adversary implementations. While typical team play emerges after 4000 epochs (~4h) of training . The evaluation results here were obtained from policies with 20k epochs (~21h) of training.

Against itself (RL), all outcomes are roughly mirrored. Spawning the ball in one team's half directly leads to dominance, which can be seen from both game outcomes and ball ownership statistics. Once they have the ball, they perform numerous passes with a very high success rate.

Against the heuristic bot, the trainee excelled across all scenarios, especially in the offensive. When averaged over all scenarios, the trainees won about $80\,\%$, drew $16\,\%$, and lost in only $4\,\%$ of all games and owning the ball most of the

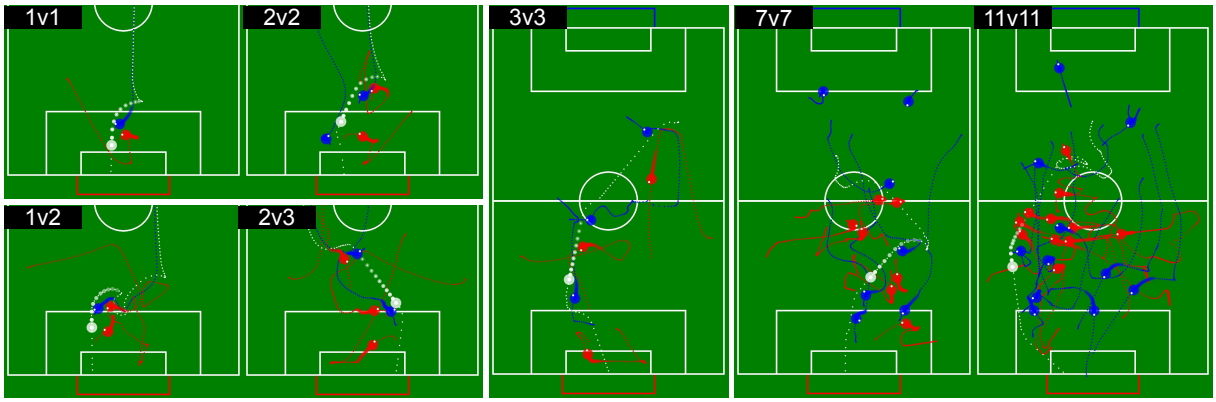


Fig. 7. Recorded motion trajectories from various games illustrating generalized cooperative behavior for diverse team composition. The big colored dots represent the current position of our trainees (blue), adversaries (red), and the ball. The last few motion frames are additionally emphasized by the fading dot trails. The doted trajectories visualize the full game dynamic. The ball (white) trajectory, in particular, is useful for understanding how the ball is passed between trainees before they score the goal at the end. Note: we used bots for lower agent scenarios. The adversaries in the 7v7 and 11v11 games are directly controlled using the same RL policy.

time. Although, more back and forth can be observed based on the pass and game duration statistics.

Finally, our trainees also prevailed against HELIOS by scoring more goals and delivering more consistent passes across all scenarios. From the significant ball ownership losses from both teams, we can conclude that both parties were actively engaging the ball. In total, our trainee won $66.8\,\%$, drew $4.2\,\%$ and lost $29\,\%$ of all games.

In addition to playing against fixed adversaries, we also conducted ablation studies to understand how dense rewards affect policy performance. We observed that the trainees failed to reach the ball without these dense rewards. Having permanent dense rewards allow the agent to learn decent gameplay, but suboptimal, winning only 12 and drawing 5 out of 100 games against the default policy.

B. Qualitative Results

We depict common trainee behaviors in Fig. 6, which provides a top-down view of a 3v3 game at different times. (A, B, and C) showcase global behavior and (A1-3, B1-3, C1-3) illustrate various local ball handling skills. Additionally, Fig. 7 showcases common behaviors in games with other team compositions. Despite encountering only teams of up to 3v3 agents during training, the trainee has learned to generalize his behavior to a full-scale 11v11 game, showcasing various quality passes despite the clustering and chaos.

Besides the policy behavior, Fig. 5 also depicts the trainees' value functions taken during a pass. It is worth noting that the ball position heat map (B1, B2) is nearly identical from both trainees' perspectives. This similarity aligns with our intuition since, unlike the base position, the ball's state is directly linked to the team's common objective, which is independent from the agent's perspective.

V. CONCLUSION

Our proposed approach allows effective end-to-end learning of cooperative behaviors in a multi-agent soccer environment and achieves an overall $66.8\,\%$ win rate against the SOTA benchmark HELIOS. By employing an improved GEE architecture, curricula, and self-play, the trainees can quickly explore and adopt intelligent behavior against previously unseen team composition (up to 11v11) and adversaries. Most of the sophisticated cooperative behavior emerges after the initial exploration, including the removal of dense rewards. With only the sparse rewards, the policy can demonstrate clear role assignment (Blue 1 in Fig. 6 A) and strategic positioning (Blue 2 and 3 in A, B). Our trainees have also acquired various ball-handling skills such as dribbling (A1-3), passing (A3, B3), and goal shooting (C3). Furthermore, the value function (Fig. 5) also clearly illustrates many of the intended strategies, such as ball assignment and positioning (A1, A2), opportunities for a pass, and goal (prominent blue funnel in B1, B2). For a more detailed illustration, we invited the readers to check out the supplementary video[5].

One limitation of our approach is corner case handling, for instance, when the ball is stuck at the border or when the opponents have the ball and are too far away. Our trainees often start drifting aimlessly in these scenarios, making them poor defenders. . Furthermore, a higher agent number in our environment frequently results in an agent cluster, sometimes stalls the game entirely. As the ultimate goal is to deploy such policies onto a robotic system, we conduct preliminary tests inside the NomadZ-NG framework [32], which simulates soccer games for NAO robots. While the movement and global positioning exhibit comparable behavior, the kick maps poorly onto humanoid robots due to the lack of continuous ball handling.

Future works should address these limitations. As our environment is already embedded in 3D, we can explore additional 3D actions such as high kicks or even directly replace our agents with fully articulated humanoid robots, possibly using our work as a hierarchical or pre-training component. Moreover, our work focuses on perfect information games where the state of all agents is available without delay. Handling additional complexity stemming from limited field of view, occlusion, and communication delay could also be an exciting research direction.

[5]https://www.youtube.com/watch?v=klETyDnWO2w

References

[1] F. Wang and K. Hauser, "Stable bin packing of non-convex 3d objects with a robot manipulator," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8698–8704.

[2] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[3] J. Lee, M. Bjelonic, A. Reske, L. Wellhausen, T. Miki, and M. Hutter, "Learning robust autonomous navigation and locomotion for wheeled-legged robots," *Science Robotics*, vol. 9, no. 89, p. eadi9641, 2024.

[4] H. Kitano, *RoboCup-97: robot soccer world cup I*. Springer Science & Business Media, 1998, vol. 1395.

[5] F. B. Martins, M. G. Machado, H. F. Bassani, P. H. Braga, and E. S. Barros, "rsoccer: A framework for studying reinforcement learning in small and very small size robot soccer," in *Robot World Cup*. Springer, 2021, pp. 165–176.

[6] S. Liu, G. Lever, Z. Wang, J. Merel, S. A. Eslami, D. Hennes, W. M. Czarnecki, Y. Tassa, S. Omidshafiei, A. Abdolmaleki, *et al.*, "From motor control to team play in simulated humanoid football," *Science Robotics*, vol. 7, no. 69, p. eabo0235, 2022.

[7] T. Haarnoja, B. Moran, G. Lever, S. H. Huang, D. Tirumala, J. Humplik, M. Wulfmeier, S. Tunyasuvunakool, N. Y. Siegel, R. Hafner, *et al.*, "Learning agile soccer skills for a bipedal robot with deep reinforcement learning," *Science Robotics*, vol. 9, no. 89, p. eadi8022, 2024.

[8] K. Kurach, A. Raichuk, P. Stańczyk, M. Zając, O. Bachem, L. Espeholt, C. Riquelme, D. Vincent, M. Michalski, O. Bousquet, *et al.*, "Google research football: A novel reinforcement learning environment," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 4501–4510.

[9] A. Smit, H. A. Engelbrecht, W. Brink, and A. Pretorius, "Scaling multi-agent reinforcement learning to full 11 versus 11 simulated robotic football," *Autonomous Agents and Multi-Agent Systems*, vol. 37, no. 1, p. 20, 2023.

[10] S. Liu, G. Lever, J. Merel, S. Tunyasuvunakool, N. Heess, and T. Graepel, "Emergent coordination through competition," *arXiv preprint arXiv:1902.07151*, 2019.

[11] B-Human Team, "B-Human Code Release," Oct. 2023. [Online]. Available: https://github.com/bhuman/BHumanCodeRelease

[12] A. Aggarwal, M. Van Der Burgh, P. Van Dooren, L. Messing, R. M. Núnez, S. Narla, G. C. Cardenas, J. Lunenburg, L. Van Beek, L. Janssen, *et al.*, "Tech united eindhoven@ home 2023 team description paper," Tech. Rep, Tech. Rep., 2023.

[13] M. Ratzel, M. Geiger, and A. Ryll, "Robocup 2023 ssl champion tigers mannheim-improved ball interception trajectories," in *Robot World Cup*. Springer, 2023, pp. 407–415.

[14] T. An, J. Lee, M. Bjelonic, F. De Vincenti, and M. Hutter, "Solving multi-entity robotic problems using permutation invariant neural networks," *arXiv preprint arXiv:2402.18345*, 2024.

[15] J. M. Catacora Ocana, F. Riccio, R. Capobianco, and D. Nardi, "Cooperative multi-agent deep reinforcement learning in soccer domains," in *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems*, 2019, pp. 1865–1867.

[16] H. Zhong, H. Zhu, and X. Li, "Development of a simulation environment for robot soccer game with deep reinforcement learning and role assignment," in *2023 WRC Symposium on Advanced Robotics and Automation (WRC SARA)*. IEEE, 2023, pp. 213–218.

[17] F. Lin, S. Huang, T. Pearce, W. Chen, and W.-W. Tu, "Tizero: Mastering multi-agent football with curriculum learning and self-play," *arXiv preprint arXiv:2302.07515*, 2023.

[18] M. Mittal, N. Rudin, V. Klemm, A. Allshire, and M. Hutter, "Symmetry considerations for learning task symmetric robot policies," *arXiv preprint arXiv:2403.04359*, 2024.

[19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[20] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 652–660.

[21] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. R. Salakhutdinov, and A. J. Smola, "Deep sets," *Advances in neural information processing systems*, vol. 30, 2017.

[22] H. Akiyama and T. Nakashima, "Helios base: An open source package for the robocup soccer 2d simulation," in *RoboCup 2013: Robot World Cup XVII 17*. Springer, 2014, pp. 528–535.

[23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[24] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Machine learning proceedings 1994*. Elsevier, 1994, pp. 157–163.

[25] S. Gronauer and K. Diepold, "Multi-agent deep reinforcement learning: a survey," *Artificial Intelligence Review*, vol. 55, no. 2, pp. 895–943, 2022.

[26] E. A. Hansen, D. S. Bernstein, and S. Zilberstein, "Dynamic programming for partially observable stochastic games," in *AAAI*, vol. 4, 2004, pp. 709–715.

[27] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, *et al.*, "Isaac gym: High performance gpu-based physics simulation for robot learning," *arXiv preprint arXiv:2108.10470*, 2021.

[28] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Conference on Robot Learning*. PMLR, 2022, pp. 91–100.

[29] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

[30] B. Brandão, T. W. De Lima, A. Soares, L. Melo, and M. R. Maximo, "Multiagent reinforcement learning for strategic decision making and control in robotic soccer through self-play," *IEEE Access*, vol. 10, pp. 72 628–72 642, 2022.

[31] "Soccer server protocols," 2024. [Online]. Available: https://rcsoccersim.readthedocs.io/en/latest/soccerserver.html#protocols

[32] NomadZ Team, "NomadZ Code Release," Oct. 2024. [Online]. Available: https://github.com/nomadz-ethz/nomadz-ng